

Automated Server Deployment Using Azure ARM Templates

Aim

To automate the deployment of a Linux-based web server in Microsoft Azure using ARM Templates, eliminating manual infrastructure provisioning and ensuring consistent, repeatable deployments.

Objectives

- Deploy Azure infrastructure using ARM Templates.
- Automatically provision networking resources such as Virtual Network, Subnet, NSG, Public IP, and NIC.
- Create an Ubuntu Virtual Machine.
- Install and configure the Nginx web server automatically using a Custom Script Extension.
- Remove the default Nginx website and deploy a custom static web application from a GitHub repository.
- Make the application publicly accessible through the VM's Public IP address.

Project Overview

This project demonstrates Infrastructure as Code (IaC) using Azure ARM Templates to provision an end-to-end web server environment. Instead of manually creating Azure resources, a single JSON template defines and deploys all required infrastructure components.

During VM provisioning, a Bash script is executed automatically through the Azure Custom Script Extension. The script performs system updates, installs Nginx and Git, clones the website source code from GitHub, replaces the default Nginx content, configures permissions, and restarts the web server.

As a result, the deployed Virtual Machine immediately serves the custom web application over HTTP using its public IP address.

Platform

- **Cloud Provider:** Microsoft Azure
- **Infrastructure as Code (IaC):** Azure Resource Manager (ARM) Templates
- **Operating System:** Ubuntu Server 18.04 LTS
- **Web Server:** Nginx
- **Version Control:** Git & GitHub

Prerequisites

Before deployment, ensure the following:

- Active Microsoft Azure subscription
- Azure Portal access
- ARM Template (ARM Templates.json)
- GitHub repository containing the website source code
- Administrator username and password for the VM

Automated Server Deployment Using Azure ARM Templates

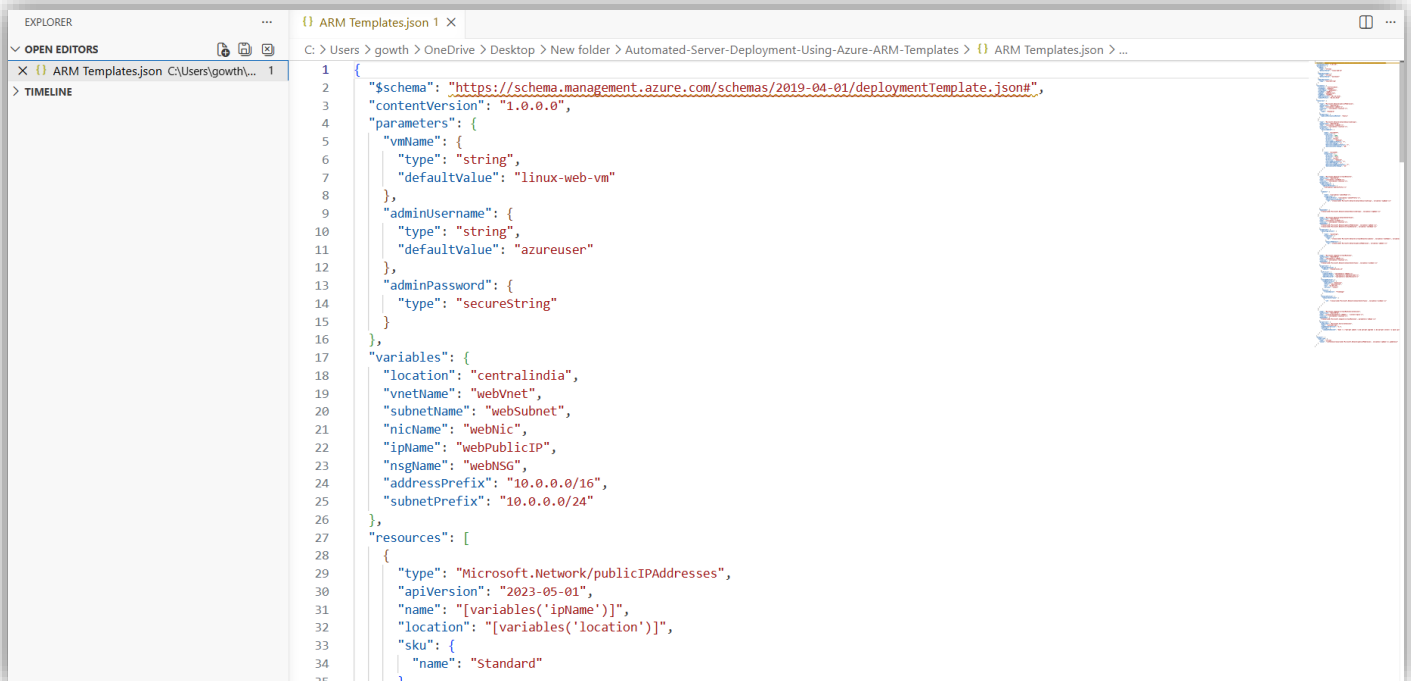
 index.html	source code
 script.js	source code
 style.css	source code

Step-by-Step Implementation

Prepare the ARM Template

Create an ARM Template (ARM Templates.json) that defines:

- Virtual Machine
- Virtual Network
- Subnet
- Network Interface
- Static Public IP Address
- Network Security Group
- Custom Script Extension

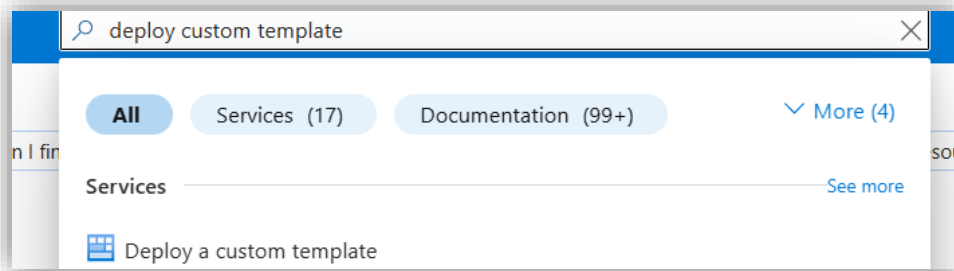


```
1 {
2   "$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentTemplate.json#",
3   "contentVersion": "1.0.0.0",
4   "parameters": {
5     "vmName": {
6       "type": "string",
7       "defaultValue": "linux-web-vm"
8     },
9     "adminUsername": {
10      "type": "string",
11      "defaultValue": "azureuser"
12    },
13    "adminPassword": {
14      "type": "secureString"
15    }
16  },
17  "variables": {
18    "location": "centralindia",
19    "vnetName": "webVnet",
20    "subnetName": "webSubnet",
21    "nicName": "webNic",
22    "ipName": "webPublicIP",
23    "nsgName": "webNSG",
24    "addressPrefix": "10.0.0.0/16",
25    "subnetPrefix": "10.0.0.0/24"
26  },
27  "resources": [
28    {
29      "type": "Microsoft.Network/publicIPAddresses",
30      "apiVersion": "2023-05-01",
31      "name": "[variables('ipName')]",
32      "location": "[variables('location')]",
33      "sku": {
34        "name": "Standard"
35      },
36    },
37    {
38      "type": "Microsoft.Network/virtualNetworks",
39      "apiVersion": "2023-05-01",
40      "name": "[variables('vnetName')]",
41      "location": "[variables('location')]",
42      "addressSpace": {
43        "addressPrefixes": [
44          "[variables('addressPrefix')]"
45        ]
46      },
47      "subnets": [
48        {
49          "name": "[variables('subnetName')]",
50          "addressPrefix": "[variables('subnetPrefix')]"
51        }
52      ]
53    },
54    {
55      "type": "Microsoft.Network/networkInterfaces",
56      "apiVersion": "2023-05-01",
57      "name": "[variables('nicName')]",
58      "location": "[variables('location')]",
59      "vnetName": "[variables('vnetName')]",
60      "subnetName": "[variables('subnetName')]",
61      "ipName": "[variables('ipName')]",
62      "adminUsername": "[parameters('adminUsername')]",
63      "adminPassword": "[parameters('adminPassword')]"
64    }
65  ]
66 }
```

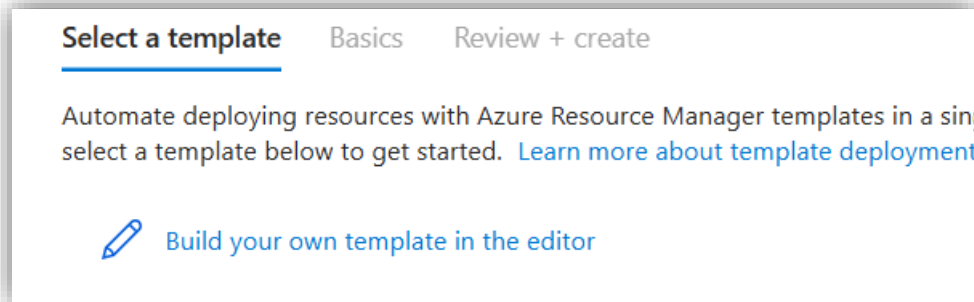
Upload the Template to Azure Portal

1. Sign in to the Azure Portal.
2. Navigate to **Deploy a Custom Template**.

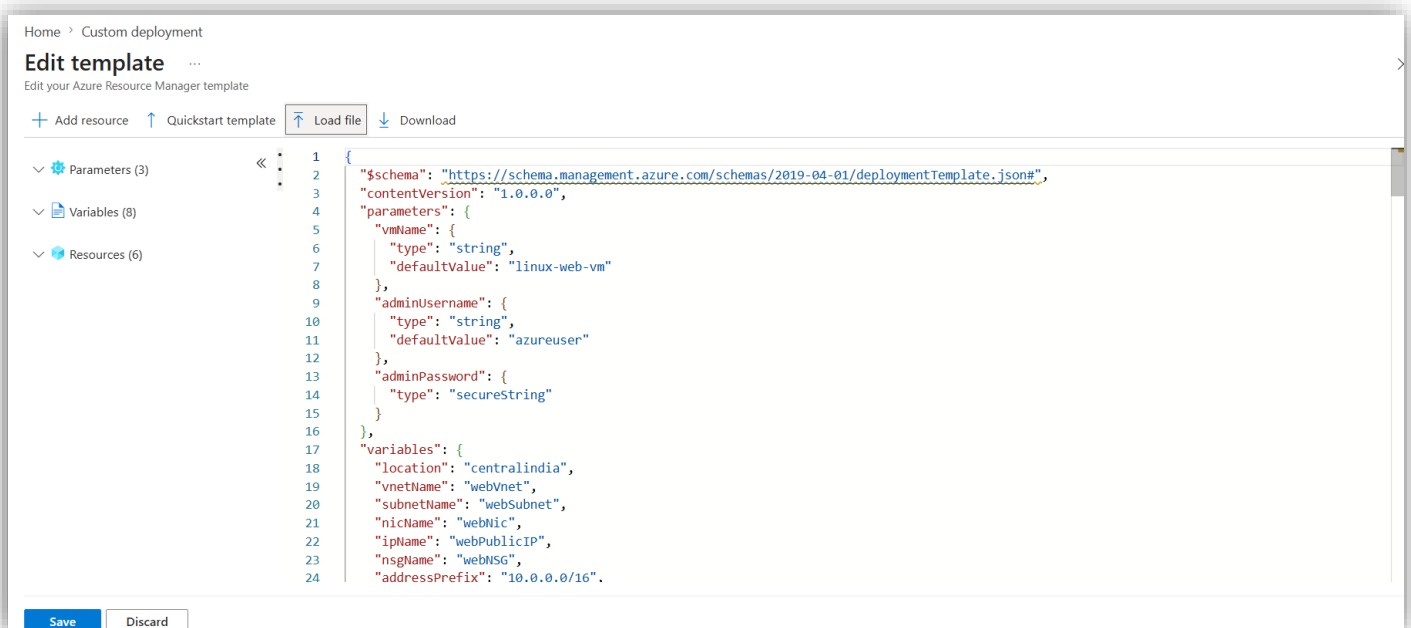
Automated Server Deployment Using Azure ARM Templates



3. Choose **Build your own template in the editor** or upload the JSON file.



4. Load the ARM Template.



Provide Deployment Parameters

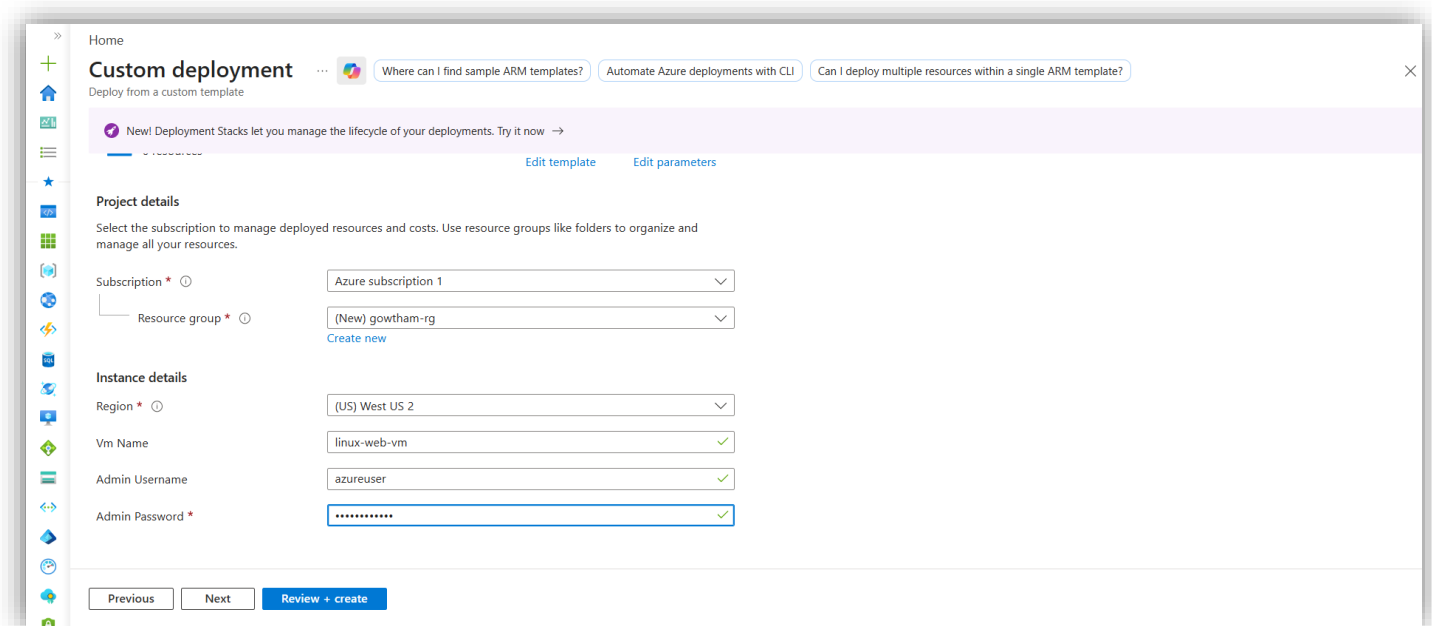
Enter values such as:

- Resource Group
- VM Name
- Administrator Username
- Administrator Password

Automated Server Deployment Using Azure ARM Templates

- Deployment Region (Central India)

Azure validates the template before deployment.



The screenshot shows the 'Custom deployment' page in the Azure Portal. The page is titled 'Custom deployment' and has a subtitle 'Deploy from a custom template'. There are three links at the top: 'Where can I find sample ARM templates?', 'Automate Azure deployments with CLI', and 'Can I deploy multiple resources within a single ARM template?'. Below these links is a purple banner with the text 'New! Deployment Stacks let you manage the lifecycle of your deployments. Try it now →'. Under the banner are two links: 'Edit template' and 'Edit parameters'. The 'Project details' section contains the following fields: 'Subscription' (set to 'Azure subscription 1'), 'Resource group' (set to '(New) gowtham-rg' with a 'Create new' link), 'Region' (set to '(US) West US 2'), 'Vm Name' (set to 'linux-web-vm'), 'Admin Username' (set to 'azureuser'), and 'Admin Password' (masked with dots). At the bottom are three buttons: 'Previous', 'Next', and 'Review + create'.

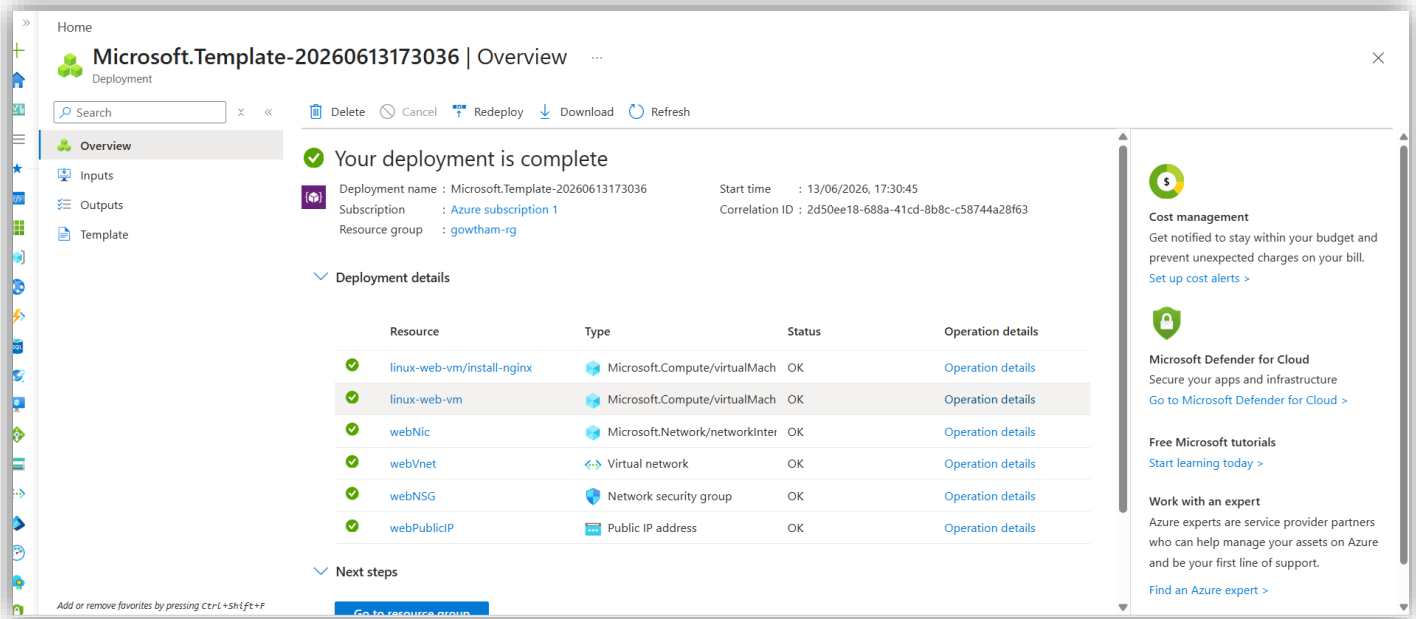
Deploy Azure Resources

Click **Review + Create** and then **Create**.

The ARM Template automatically provisions:

- Static Public IP
- Network Security Group
- Virtual Network
- Subnet
- Network Interface
- Ubuntu Virtual Machine
- VM Extension

Automated Server Deployment Using Azure ARM Templates



Network Security Configuration

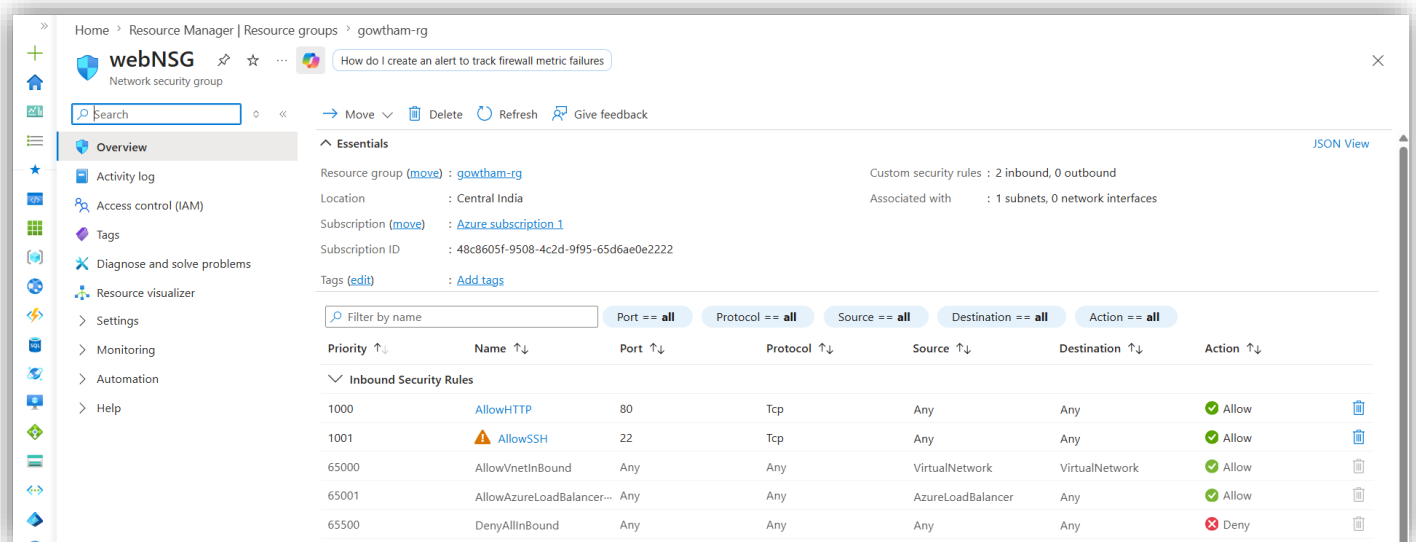
The template creates a Network Security Group (NSG) with inbound rules allowing:

Protocol Port Purpose

TCP 80 HTTP access for the website

TCP 22 SSH access for administration

This enables public access to the web application and remote management of the server.



Virtual Machine Provisioning

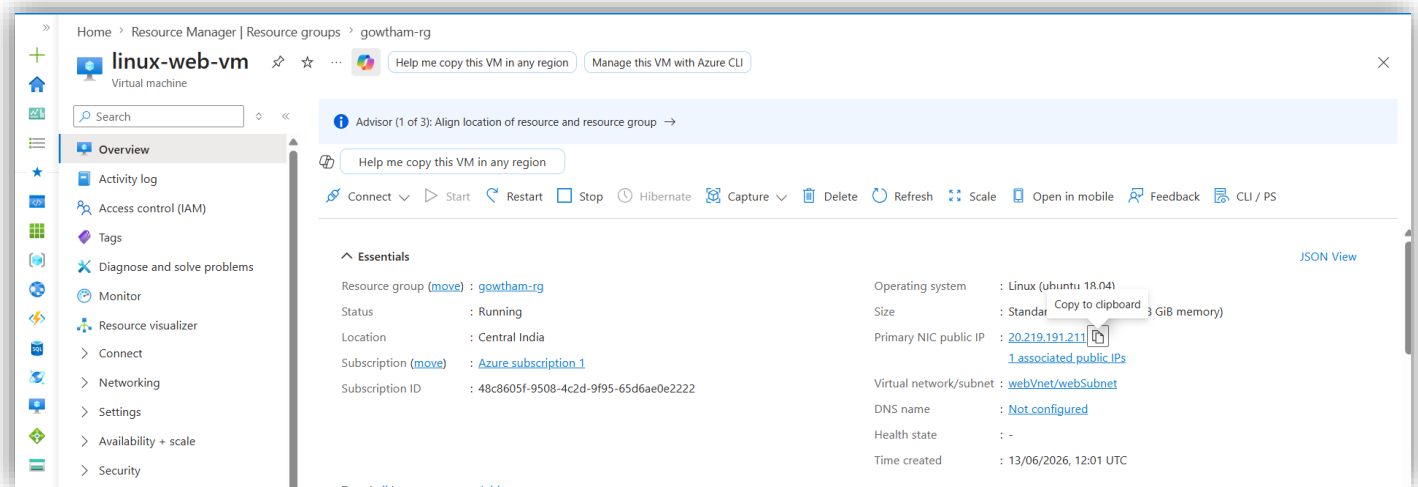
An Ubuntu 18.04 LTS Virtual Machine is created with:

- VM Size: Standard_D2s_v3
- Static Public IP
- Attached Network Interface

Automated Server Deployment Using Azure ARM Templates

- Virtual Network connectivity

The VM serves as the hosting environment for the web application.



Automatic Server Configuration Using Bash Script

After VM creation, the Azure Custom Script Extension executes a Bash script that performs automated server configuration.

The script executes the following commands:

```
apt-get update -y
apt-get upgrade -y
apt-get install -y nginx git

systemctl enable nginx
systemctl start nginx

rm -rf /var/www/html/*

git clone https://github.com/gowthamsaikadali/myntna-clone.git /tmp/myntna-clone

cp -r /tmp/myntna-clone/* /var/www/html/

chmod -R 775 /var/www/html
chgrp -R azureuser /var/www/html

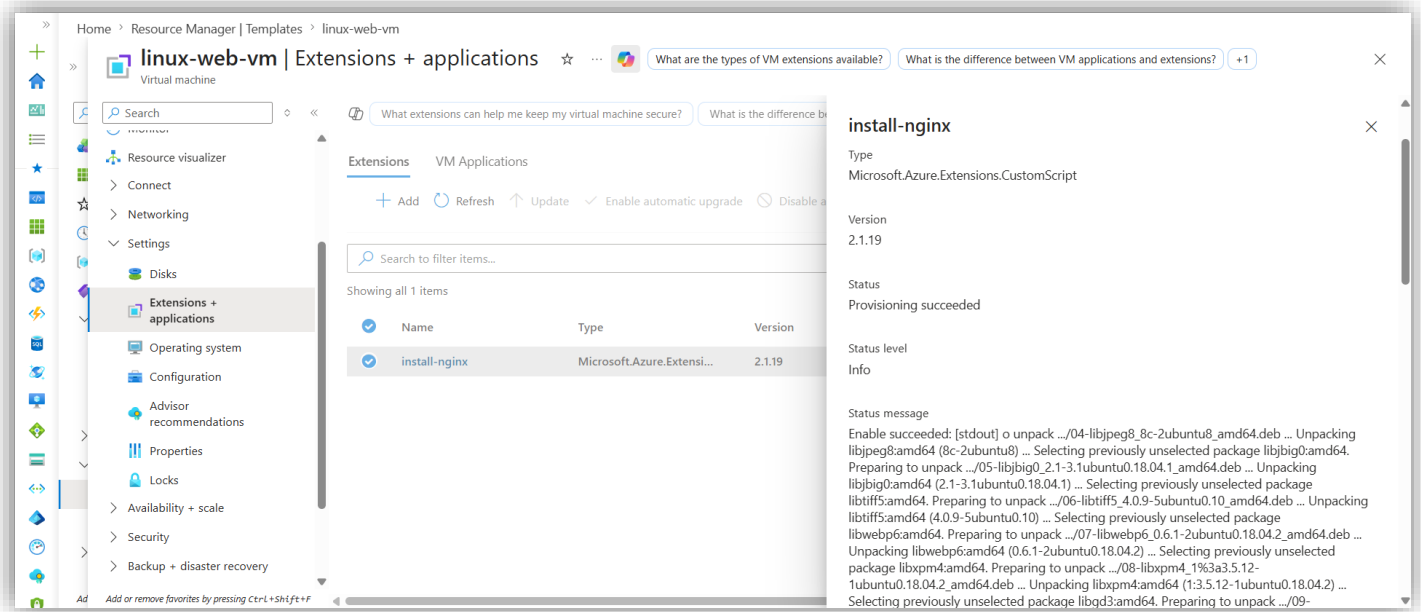
systemctl restart nginx
```

Actions Performed

- Updates package repositories.
- Upgrades installed packages.
- Installs **Nginx** and **Git**.
- Enables and starts the Nginx service.
- Removes the default website from /var/www/html.
- Clones the custom web application from GitHub.

Automated Server Deployment Using Azure ARM Templates

- Copies the application files into the Nginx web root.
- Sets appropriate permissions and group ownership.
- Restarts Nginx to serve the new content.



Website Deployment

The website source code is automatically downloaded from:

<https://github.com/gowthamsaikadali/myntna-clone>

The contents are copied to:

`/var/www/html/`

This replaces the default Nginx landing page with the custom static website.

 <code>index.html</code>	source code
 <code>script.js</code>	source code
 <code>style.css</code>	source code

Access the Application

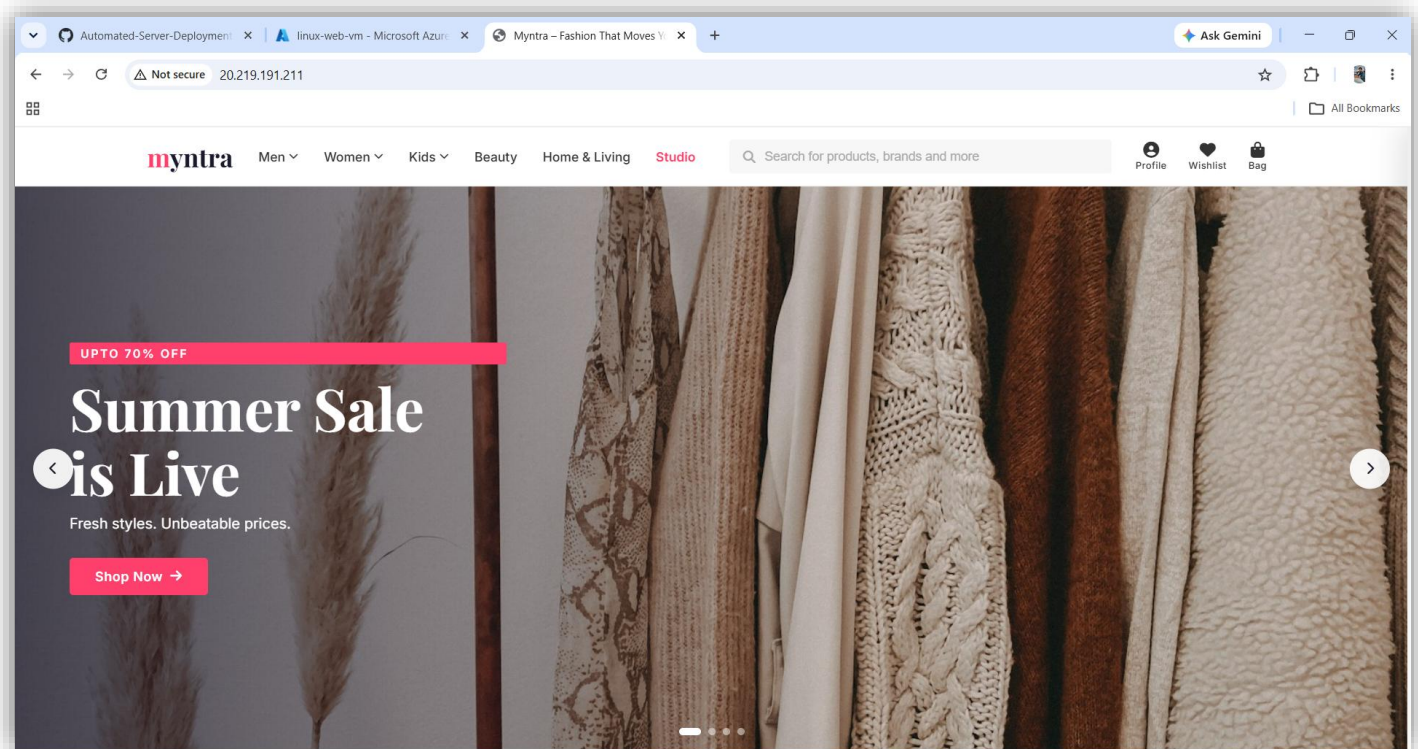
After deployment completes:

1. Open the Azure Portal.
2. Navigate to the Virtual Machine.
3. Copy the assigned Public IP Address.
4. Open a web browser.
5. Enter:

Automated Server Deployment Using Azure ARM Templates

<http://<Public-IP>>

The deployed static website should load successfully.



Automation Workflow

1. ARM Template deployment starts.
2. Azure provisions networking resources.
3. The Ubuntu Virtual Machine is created.
4. The Custom Script Extension executes the Bash script.
5. Nginx and Git are installed.
6. The default Nginx content is removed.
7. The website repository is cloned from GitHub.
8. Website files are copied to /var/www/html.
9. Nginx is restarted.
10. The web application becomes accessible via the VM's public IP.

Outcome

The project successfully automated the deployment of a web server in Azure using ARM Templates. A fully configured Ubuntu VM with Nginx was provisioned without manual server setup, and the custom static website was automatically deployed from a GitHub repository. The application became accessible through the assigned public IP address, demonstrating the effectiveness of Infrastructure as Code for repeatable and consistent cloud deployments.

Conclusion

This project showcases how Azure ARM Templates can be used to automate infrastructure provisioning and application deployment. By combining declarative resource definitions with a Custom Script Extension, the

Automated Server Deployment Using Azure ARM Templates

solution creates a complete web hosting environment with minimal manual intervention. The approach improves deployment consistency, reduces configuration errors, and provides a scalable foundation for future cloud-native projects and DevOps practices.